

On Task Type Diversity

Document Number: D4050R0
Date: 2026-03-11
Audience: LEWG
Reply-to: Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. Task Types in Production

3. Why the Invariants Differ

3.1 Frame Allocator

3.2 Foreign Awaitables

3.3 Timing

4. Cross-Domain Bridges

5. Recommendation

6. Suggested Straw Polls

Acknowledgements

References

WG21 Papers

Libraries

Abstract

Every production C++ coroutine library ships its own task type. Six libraries independently converged on a one-parameter declaration. Each extends the C++20 awaitable protocol with domain-specific safety invariants - frame allocator propagation for networking, device memory contracts for GPU, zero-allocation guarantees for high-frequency trading. Cross-domain composition requires explicit bridges that translate these invariants. This paper surveys the evidence, documents why the invariants differ by design, and recommends that `std::execution::task` (P3552R3^[1]) be documented as the standard task type for the sender domain rather than presented as a general-purpose task type for all coroutine users.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.
-

1. Disclosure

The author developed P4003R0^[2] (“Coroutines for I/O”), P4007R1^[3] (“Senders and Coroutines”), P4014R0^[4] (“The Sender Sub-Language”), and P2583R1^[5] (“Symmetric Transfer and Sender Composition”). A coroutine-only design cannot express compile-time work graphs, does not support heterogeneous dispatch, and assumes a cooperative runtime. Those are real costs.

The frame allocator propagation gap described in Section 3 was identified by Peter Dimov. The cross-library bridge evidence in Section 4 was produced by Klemens Morgenstern, the author of Boost.Cobalt. Neither is a co-author of this paper or any paper in this series.

2. Task Types in Production

Six production coroutine libraries independently converged on a one-parameter task type.

Library	Declaration	Params	Source
asyncpp	<code>template<class T> class task</code>	1	<code>task.hpp</code>

Library	Declaration	Params	Source
Boost.Cobalt	<code>template<class T> class task</code>	1	<code>task.hpp</code>
cppcoro	<code>template<typename T> class task</code>	1	<code>task.hpp</code>
aiopp	<code>template<typename Result> class Task</code>	1	<code>task.hpp</code>
libcoro	<code>template<typename return_type> class task</code>	1	<code>task.hpp</code>
folly::coro	<code>template<typename T> class Task</code>	1	<code>Task.h</code>
P3552R3 (std)	<code>template<class T, class Environment> class task</code>	2	<code>task.hpp</code>

The external interface converged. The internal machinery diverged. Boost.Cobalt embeds intrusive linked list elements in the promise for its cancellation model. folly::coro::Task integrates with Facebook's fiber scheduler and `CancellationToken`. Boost.Asio's `awaitable` is coupled to `io_context`. cppcoro targets platform-specific I/O primitives. Each library optimizes for its domain.

P3552R3^[1] adds a second template parameter: `Environment`. This means `std::execution::task<int>` and `std::execution::task<int, MyEnvironment>` are different types. The sender environment - which every production library independently chose not to expose in the type signature - is now part of the public type.

A networking task type can use thread-local propagation for frame allocators. A GPU task type can use device memory APIs. A high-frequency trading task type can enforce zero allocation at compile time. Solutions exist when the promise is free to serve its domain rather than forced to serve a model it does not participate in.

The diversity in the table above is not fragmentation. It is fitness.

3. Why the Invariants Differ

Every production coroutine library extends the base C++20 awaitable protocol. The extensions enforce domain-specific safety invariants. P3552R3^[1]'s `await_transform` + `affine_on` maintains executor continuity and stop token propagation for recognized senders. Three gaps remain, each independently documented.

3.1 Frame Allocator

`affine_on` returns the coroutine to the correct scheduler thread after completion. The thread-local frame allocator value on that thread is whatever was left by the previous coroutine that executed on it - not the allocator for this chain. There is no hook between `affine_on` completing and the coroutine body resuming

where TLS can be restored. Peter Dimov identified this gap. It is unsolved in the published record: [D3980R0](#)^[6] addresses allocator sourcing from the receiver environment but does not restore TLS on resumption.

3.2 Foreign Awaitables

[P3552R3](#)^[1]'s `await_transform` recognizes senders. If a user `co_await` s a non-sender awaitable - a `cppcoro::task<int>`, a Boost.Cobalt awaitable, any basic C++20 awaitable - it passes through without `affine_on`, without stop token wiring, without any environment propagation. All three invariants are lost. Networking libraries reject non-conforming awaitables at compile time because permissiveness breaks their safety invariants. [P3552R3](#)^[1] permits the permissive case.

3.3 Timing

The sender extension delivers context through `connect / start` after the coroutine frame is allocated. The networking extension delivers context at `co_await` time through the `await_suspend` parameter. The frame allocator must be available before the child's `operator new` fires, and `operator new` fires at the call site - before any sender machinery runs. No mechanism in `std::execution` delivers any allocator to `operator new` without `allocator_arg_t` in the parameter list. Jonathan Müller confirmed the symmetric transfer consequence in [P3801R0](#)^[7]: *“a thorough fix is non-trivial and requires support for guaranteed tail calls.”* Dietmar Kühl documented the gap as an open issue in [P3796R1](#)^[8].

4. Cross-Domain Bridges

The [cross_await](#)^[9] repository, authored by Klemens Morgenstern, contains four working cross-library composition examples. None compose natively through the base C++20 protocol alone. All four require explicit adaptor code.

Outer library	Inner library	Adaptor file	Lines
Cobalt	Capy	<code>cobalt_capy.cpp</code>	67
TMC	Capy	<code>tmc_capy.cpp</code>	51
Capy	Cobalt	<code>corosio_cobalt.cpp</code>	105
Capy	TMC	<code>corosio_tmc.cpp</code>	60

Each adaptor translates execution context between two domain protocols. From `cobalt_capy.cpp` :

```

template<typename T>
struct capy_task_adaptor {
    capy::task<T> tt;
    capy::io_env env;
    std::stop_source src;
    boost::asio::cancellation_slot slt;

    template<typename Promise>
    auto await_suspend(std::coroutine_handle<Promise> h) {
        auto& p = h.promise();
        env.executor      = p.get_executor();
        env.stop_token    = src.get_token();
        env.frame_allocator = p.get_allocator().resource();
        slt = p.get_cancellation_slot();
        slt.assign([this](auto ct) {
            if ((ct & boost::asio::cancellation_type::terminal) !=
                boost::asio::cancellation_type::none)
                src.request_stop();
        });
        return tt.await_suspend(h, &env);
    }
};

```

The bridge knows both Cobalt’s internal API and Capy’s protocol. It is specific to the pair of libraries. There is no generic bridge because the invariants being translated are domain-specific. Cross-domain composition is achievable but requires library authors with knowledge of both protocols. The C++20 awaitable protocol is the surface all four bridges bottom out to.

5. Recommendation

`std::execution::task` is correctly positioned in `namespace std::execution`. The committee should not present it as the general-purpose coroutine task type in tutorials, examples, or guidance. Domain-specific task types are the norm in production and should be recognized as the intended model for coroutine composition. The design space for coroutine-to-execution-context integration is open. The committee should not foreclose it.

6. Suggested Straw Polls

1. "Domain-specific task types that extend the C++20 awaitable protocol are the intended model for coroutine composition in C++."
 2. " `std::execution::task` should be documented as the standard task type for the sender domain, not as a general-purpose task type for all coroutine users."
-

Acknowledgements

The authors would like to thank Peter Dimov for identifying the frame allocator propagation gap and requiring the `IoAwaitable` fix that clarified the relationship between domain-specific invariants and the base protocol; Klemens Morgenstern for authoring Boost.Cobalt and producing the `cross_await` bridge evidence that demonstrates cross-domain composition in practice; Dietmar Kühl for authoring [P3552R3](#) and [P3796R1](#), whose open documentation of task issues reflects the thoroughness the committee depends on; Jonathan Müller for independently confirming the symmetric transfer gap in [P3801R0](#); and Mungo Gill, Mohammad Nejati, and Michael Vandenberg for their feedback during development.

References

WG21 Papers

1. [P3552R3](#) - "Add a Coroutine Task Type" (Dietmar Kühl, Maikel Nadolski, 2025). <https://wg21.link/p3552r3>
2. [P4003R0](#) - "Coroutines for I/O" (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). <https://wg21.link/p4003r0>
3. [P4007R1](#) - "Senders and Coroutines" (Vinnie Falco, Mungo Gill, 2026). <https://wg21.link/p4007r1>
4. [P4014R0](#) - "The Sender Sub-Language" (Vinnie Falco, 2026). <https://wg21.link/p4014r0>
5. [P2583R1](#) - "Symmetric Transfer and Sender Composition" (Mungo Gill, Vinnie Falco, 2026). <https://wg21.link/p2583r1>
6. [D3980R0](#) - "Task's Allocator Use" (Dietmar Kühl, 2026). <https://isocpp.org/files/papers/D3980R0.html>
7. [P3801R0](#) - "Concerns about the design of `std::execution::task`" (Jonathan Müller, 2025). <https://wg21.link/p3801r0>
8. [P3796R1](#) - "Coroutine Task Issues" (Dietmar Kühl, 2025). <https://wg21.link/p3796r1>

Libraries

9. Klemens Morgenstern, **cross_await** - “co_await one coroutine library from another” (2026).
https://github.com/klemens-morgenstern/cross_await
10. **cppcoro** - C++ coroutine library (Lewis Baker). <https://github.com/lewissbaker/cppcoro>
11. **libcoro** - C++20 coroutine library (Josh Baldwin). <https://github.com/jbaldwin/libcoro>
12. **asyncpp** - Async coroutine library (Péter Kardos). <https://github.com/petiaccja/asyncpp>
13. **aiopp** - Async I/O library (Joel Schumacher). <https://github.com/pfirsich/aiopp>
14. **beman::task** - P3552R3 reference implementation. <https://github.com/bemanproject/task>
15. **folly::coro** - Facebook coroutine task type.
<https://github.com/facebook/folly/blob/main/folly/experimental/coro/Task.h>
16. **Boost.Cobalt** - Boost coroutine task type (Klemens Morgenstern).
<https://github.com/boostorg/cobalt/blob/develop/include/boost/cobalt/task.hpp>
17. **Capy** - Coroutine-native I/O library (Vinnie Falco). <https://github.com/cppalliance/capy>